

Ingeniería del Software 3

Defensa P1

Trabajos Prácticos 1 a 4

Individual · oral · en tu notebook · con reloj

UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

Dos grupos, dos fechas

Grupo B

34 alumnos

Defensa

Miércoles 2/9

de 16:00 a 19:30

El formulario cierra

Lunes 31/8 · 23:59

6:00 por persona, exactos

Grupo A

40 alumnos

Defensa

Viernes 4/9

de 13:30 a 17:00

El formulario cierra

Miércoles 2/9 · 23:59

5:15 por persona, exactos

Grupos y horarios DEFINITIVOS. Fijate en qué grupo quedaste, en qué puesto y a qué hora: la lista de cada grupo está publicada en el repositorio de la materia, con el orden sorteado.

Cómo se arma la nota de P1

25 %

Configuración técnica

Lo que hay en tu repo público al cierre:
protecciones, PRs, Dockerfiles,
compose, Project, workflow, badge...

Se corrige ANTES, sobre tu repo.

25 %

decisiones.md

Tu documento, sección por TP: por qué
elegiste cada cosa, qué problemas
tuviste, qué hizo la IA.

Se lee ANTES.

50 %

Defensa oral

Tu turno en la mesa: mostrás los cuatro
TPs y respondés cinco preguntas.

Es lo ÚNICO que se pone el día de la
defensa.

Se aprueba con 4. Lo de antes no se discute en la mesa: la mesa es para verte navegar tu repo y explicarlo.

“Si no lo podés explicar, no lo aprobás — aunque funcione.” Un problema bien contado no resta.

decisiones.md: sin él, no hay nota

1

Se revisa antes

La corrección previa busca tu decisiones.md en main y una sección por TP. Lo que no está, no existe.

2

Lo leo yo

Con eso pongo el 25 % de decisiones y armo las preguntas que te voy a hacer en la mesa.

3

Si falta un TP

Ese TP vale 0 en decisiones. No lo puedo evaluar de lo que no está escrito.
El TP4 pesa 45 %.

Es UN archivo en la raíz del repo que se acumula: ## TP1 · ## TP2 · ## TP3 · ## TP4. Cada sección con lo que pide el enunciado de ese TP (siguiente filmina). **De las 5 entregas ya cargadas, en 3 el decisiones.md tiene la sección de un solo TP.**

Se corrige lo que está en main: si tu decisiones.md quedó en un Pull Request sin mergear, no existe. Mergealo.

Primero: cargá el formulario

5 de 74

cargaron el formulario al 26/8

Qué se carga: la URL de tu repositorio público y la URL de tu Project (TP3).

Cuándo: ya. Podés volver a abrir tu respuesta y corregirla.

Qué se corrige: lo que hay en main la mañana siguiente al cierre — no lo que había cuando lo cargaste.

Si no está cargado al cierre: **no se pre-corrige. Nota técnica 0.**

Formulario P1 (también en el README de la cátedra y en el grupo de WhatsApp):

<https://docs.google.com/forms/d/e/1FAIpQLSfDN9ytzgGD9RzPu9TDSG1REWhDY-uqlQroKnMWzcJGFArkpQ/viewform>

URL del Project — Sí: `github.com/users/<vos>/projects/3` **NO:** `github.com/<vos>/mi-app` (eso es un repo)

El día de la defensa, paso a paso

1 · Llegás

Con la notebook cargada y el sistema YA corriendo. Ubicás tu turno en la lista.



2 · Banquillo

Cuando llamo al que está antes que vos, te sentás al lado con todo abierto.



3 · Tu turno

Es individual: vos solo, con tu notebook. Timer a la vista con tus minutos (los mismos para todos los de tu fecha). Vos conducís: los cuatro TPs en orden, sin que te lo pida. Yo pregunto.



4 · Se corta

Cuando se acaban tus minutos se corta lo que esté. Yo anoto 0, 1 o 2 por cada TP y por la pregunta sobre tu caso — cinco casillas — y pasa el siguiente.

Sin proyector: se mira en tu pantalla, yo me siento al lado.

Orden de llamado: sorteado, se publica el jueves 27/8 a la noche con los grupos definitivos. La clase puede terminar más tarde de lo habitual: si te toca al final, contá con eso.

Si un movimiento no te sale: pasamos al siguiente. Si algo se rompe, el reloj sigue corriendo: no se frena para arreglarlo.

Si faltás: P1 queda sin presentar y lo recuperarás en la clase 12, presentando el bloque completo.

Cuando te sentás, esto ya tiene que estar listo

☐ TP1 — Git

Una terminal parada dentro del repo, con tu usuario de GitHub ya autenticado (vas a hacer un push). Pestaña: tu repo en GitHub, con el PR del conflicto y tu release a mano.

☐ TP2 — Docker

El sistema YA corriendo: `docker compose -f docker-compose.registry.yml up -d`, con el `.env` copiado de tu `.env.example`. El compose que BAJA las imágenes, no el que las construye. Nada se levanta en frío en la mesa.

☐ TP3 — Planificación

Pestaña: tu Project, parado en una tarea cerrada (la que vas a mostrar con su PR y su commit).

☐ TP4 — CI

Pestañas: el PR que quedó en rojo y después en verde · la pestaña Actions · el README con el badge.

☐ Batería o cargador. ☐ El guion ensayado en casa con reloj, en 6:00 si estás en el B o 5:15 si estás en el A.

Si llegás sin esto, el tiempo de abrirlo es tuyo.

Tu turno, de principio a fin

empezás

se corta



Tu turno dura 6:00 en el grupo B (16:00 a 19:30) y 5:15 en el grupo A (13:30 a 17:00). Sale de dividir las 3 h 30 de clase por los que quedaron en cada grupo. **Ensayá el guion con reloj en ese tiempo.**

El reparto sigue el peso de cada TP en la nota: TP1 5 % · TP2 40 % · TP3 10 % · TP4 45 %. Lo que más pesa es lo que más tiempo tiene.

Son los cuatro movimientos del cierre de la Clase 4. Vos los mostrás en ese orden, sin que te lo pida; en cada uno te hago UNA pregunta corta.

Qué mostrás y qué te pregunto

TP	Mostrás (vos)	Te pregunto (yo)
TP1	Un commit local sin subir + git push → rechazado por la protección. Tu tag y tu release.	por qué el flujo de trabajo está armado así
TP2	docker compose -f docker-compose.registry.yml ps → IMAGE = tus imágenes publicadas (ghcr.io/... o Docker Hub), la base en STATUS healthy. Después, en GitHub: está .env.example y NO está .env, y tu .gitignore tiene la línea .env; y docker volume ls	por qué el sistema está armado así y qué pasa si le falta una pieza
TP3	En el Project: una tarea cerrada → el PR que la cerró → el commit → su historia y su épica.	cómo se conecta lo que planificaste con lo que hiciste
TP4	El PR con el check en rojo → merge bloqueado → el fix → verde → mergeado. El badge en el README.	qué frena un cambio y qué lo destraba
Tu caso	— (no mostrás nada)	UNA pregunta sobre algo concreto de TU repo o de TU decisiones.md: un problema que contaste, una decisión que tomaste, un commit raro. Sólo la responde quien estuvo ahí.

Tu guion, paso por paso – ensayalo así en casa

#	TP	Hacés	Y decís
1	TP1	Con un cambio sin subir: <code>git commit -am "prueba"</code> y <code>git push</code> → lo rechaza la protección.	por qué lo rechaza
2	TP1	En GitHub: tu release (el tag) y el PR donde resolviste el conflicto.	qué es cada cosa
3	TP2	<code>compose -f docker-compose.registry.yml ps</code> (ya levantado): IMAGE con tus imágenes, base healthy.	de dónde vienen esas imágenes
4	TP2	En GitHub: está <code>.env.example</code> y NO está <code>.env</code> , y tu <code>.gitignore</code> tiene la línea <code>.env</code> . En la terminal: <code>docker volume ls</code> .	dónde vive cada cosa
5	TP3	Project → una TAREA cerrada → su PR → su commit → y subís a la historia y la épica.	cómo se conectan
6	TP4	El PR rojo → merge bloqueado → el fix → verde → mergeado. Actions y el badge.	qué lo bloqueó y qué lo destrabó
7	Tu caso	Nada que mostrar: te hago UNA pregunta sobre tu repo o tu decisiones.md.	lo que viviste

Después del paso 1, deshacés el commit de prueba: `git reset --hard HEAD~1`

¿Hiciste el riel Azure DevOps? Mismo guion con lo tuyo: Repos (push rechazado por la policy), Boards (work item cerrado → PR → commit) y Pipelines (build rojo → verde). Avisame antes de tu turno.

Ensayalo con reloj: 6:00 si estás en el B, 5:15 si estás en el A.

Cómo se puntúa: cinco casillas, 0 · 1 · 2

0

No lo mostrás ni lo explicás

1

Lo mostrás, pero no contestás la pregunta

2

Lo mostrás y contestás la pregunta

Un ejemplo, con el TP2

Mostrás compose ps y se ven tus imágenes de ghcr.io → ya tenés 1. **Te pregunto de dónde salieron.** Si contestás “las construí con mi Dockerfile y las subí con docker push” → 2. Si decís “no sé, las bajé de algún lado” → 1. Si el sistema no está levantado y tampoco sabés qué debería verse → 0.

Las cinco casillas suman hasta 10 puntos: eso es tu oral. Tu P1 = 25 % configuración técnica + 25 % decisiones.md + 50 % oral. Se aprueba con el 60 % del total (que es un 4).

Si algo no te arranca por la máquina pero lo sabés explicar, es 1, no 0. ¿Ansiedad o algo que necesites adaptado? Escribime antes: se acomoda sin que tengas que contarlo en la mesa.

Lo que vi en las entregas: corregilo

Todos · decisiones.md, 4 TPs

Es UN archivo que se acumula, una sección por TP. Sin sección = 0 en ese TP. El TP4 pesa 45 %.

Todos · PR abierto no cuenta

Se corrige main. Tu declaración de IA en un PR sin mergear no existe. Mergealo.

TP2 · app en el repo del TP1

TP2 va en el MISMO repo del TP1 (por PR). En otro repo la corrección no la ve.

TP2 · registry sin build:

En la mesa se levanta docker-compose.registry.yml con tus imágenes publicadas. Si IMAGE dice <carpeta>-backend, no vale.

TP3 · URL del Project

Sí: github.com/users/<vos>/projects/3
NO: github.com/<vos>/mi-app (es un repo).
Sin Project no hay TP3.

TP4 · workflows con “s”

En .github/workflow/ GitHub nunca lo corre: Actions vacío. Y es runs-on: ubuntu-latest. Abrí un PR y mirá que aparezca el check.

Vale para todos: si no entregaste todavía, te sirve para no caer en lo mismo.

Esto también se mira al corregir: revisalo

Todos · el formulario

Repo renombrado → actualizá la URL.
Campo “compañero”: SIN COMPAÑERO.
(Linus Torvalds no te revisa los PRs.)

Todos · Markdown

Título con espacio después del #. Si no, GitHub no lo muestra como título. Mirá tu archivo en GitHub, no sólo en tu editor.

Todos · el historial dice cuándo trabajaste

Los cuatro TPs en una tarde se ve, y se pregunta. Un TP que entra en UN solo commit “traído de otro lado” no tiene historia: preparete para contarla.

Todos · IA: declarala y entendela

Qué hizo la IA, qué hiciste vos, cómo lo verificaste. Si la app la escribió una IA, perfecto: la pregunta sobre tu caso va a ser sobre una línea de ese código.

TP3 · historia ≠ tarea

“Como desarrollador quiero guardar los datos...” sigue siendo una tarea. El usuario de una historia es quien recibe valor, no quien programa.

TP3 · sprint y WIP con porqué

“3 semanas para no complicarme” es al revés de lo que vimos. La duración y el límite se justifican, no se eligen por comodidad.

Nada de esto suma nota por sí solo, pero todo esto se ve en la corrección y se pregunta en la mesa.

decisiones.md — qué tiene que decir cada sección

TP	Lo que se lee (está en el enunciado de cada TP)
TP1	Por qué Git no pudo resolver el conflicto solo y qué habría tenido que pasar para que no apareciera · problemas y cómo los resolviste · IA. (Y el evidencias.md del TP1, con sus cuatro capturas.)
TP2	Por qué esa app (los cuatro criterios) · por qué dos etapas · cómo se encuentran los servicios · healthcheck vs depends_on · dónde viven los secretos · problemas · IA.
TP3	Duración del sprint con su porqué · límite de WIP y por qué ese número · la historia mal escrita: por qué está mal y cómo la reescribirías · problemas · IA.
TP4	Por qué esos jobs y por qué en paralelo · qué se cachea y qué pasa si el cache desaparece · por qué construye con tu Dockerfile · problemas · IA.

Tropiezos bien contados valen más que un camino perfecto: son los que muestran que entendiste.

Qué tenés que hacer ahora — en orden

1 Hoy

Cargá el formulario con la URL de tu repo y la de tu Project. Aunque te falten cosas: se corrige lo que haya al cierre, no lo que había cuando lo cargaste.

2 Hoy

Buscá tu nombre en la lista de tu grupo (está en el repositorio de la materia): anotate tu puesto y tu hora.

3 Antes del cierre

Completá tu decisiones.md con las cuatro secciones (TP1, TP2, TP3, TP4) y mergealo a main — o sea, que el cambio entre por Pull Request y quede en la rama principal, no en un PR abierto.

4 Antes del cierre

Revisá la lista de correcciones de las filminas anteriores sobre TU repo.

5 Antes de tu defensa

Ensayá el guion completo con reloj en tu máquina: 6:00 si estás en el B, 5:15 si estás en el A.

6 El día

Llegá con todo abierto y el sistema ya corriendo (filmina “Cuando te sentás”).

Si faltás, P1 queda sin presentar y lo recuperarás en la clase 12 con el bloque completo.

Diccionario rápido — por si alguna palabra te frena

Palabra	Qué quiere decir	Palabra	Qué quiere decir
main	La rama principal de tu repo: lo que se corrige.	imagen / registry	El paquete con tu app ya construida, y el lugar de internet donde la publicaste (ghcr.io o Docker Hub).
PR (Pull Request)	El pedido de meter un cambio en main, con su conversación.	volumen	El cajón donde Docker guarda los datos de la base para que no se pierdan.
mergear	Aceptar ese pedido: el cambio entra a main. Hasta que no lo mergeás, tu cambio no cuenta.	healthy	Estado de un contenedor que respondió bien al chequeo de salud.
tag / release	La etiqueta que congela una versión (v1.0.0) y su publicación en GitHub.	workflow / Actions	El archivo que dice qué se ejecuta solo en cada cambio, y la pestaña donde se ven esas corridas.
historia / tarea / épica	Lo que planificaste en el Project: la épica agrupa historias, cada historia tiene tareas.	badge	La chapita de color en el README que dice si la última corrida salió bien.

Si alguna palabra de estas filminas no te cierra, preguntala hoy — no el día de la defensa.

Si no lo podés explicar, no lo aprobás — aunque funcione.

Un problema bien contado no resta.

Grupo B: formulario cierra Lunes 31/8 · 23:59 · defensa Miércoles 2/9

Grupo A: formulario cierra Miércoles 2/9 · 23:59 · defensa Viernes 4/9

Buscá tu puesto y tu hora en la lista de tu grupo, publicada en el repositorio de la materia.

decisiones.md con los 4 TPs en main al cierre: sin eso no hay nota de decisiones.

Cargá el formulario hoy. Ensayá el guion con reloj: 6:00 en el B, 5:15 en el A.